

组号：_____

本科实验报告

TurtleBot4 移动机器人自主导航与避障

Experiment 2: Autonomous Navigation of TurtleBot4

课程名称：_____ 机器人与智能系统综合实践

姓 名：_____ 请填写

院 系：_____ 请填写

专 业：_____ 请填写

学 号：_____ 请填写

指导老师：_____ 请填写

实验平台：_____ ROS 2 Humble / TurtleBot4

2026 年 7 月 14 日

摘 要

本实验面向 TurtleBot4 移动机器人，在 Ubuntu 22.04 与 ROS 2 Humble 环境下构建 Ignition Gazebo 仿真系统，完成环境建图、基于静态地图的自定位、代价感知 A* 全局路径规划以及 Nav2 闭环自主导航。针对初版“建图范围小、路径短、雷达云漂移”的反馈，在原系统上增加扩展建图路线、地图覆盖率门槛、AMCL/TF/LaserScan 稳定性检查以及长距离导航 action 验收，形成可自动判定的证据链。

扩展 SLAM 验收中，机器人累计行程达到 5.00 m，已知栅格增加 58,506 个，分别满足不少于 5.0 m 与 15,000 个栅格的门槛。定位前置检查连续观察 15 s，实测 `map -> base_link` 位置漂移为 0.000 m、航向漂移为 0.000 rad，LaserScan 端点与地图占用栅格匹配率为 96.5%，明显高于 35% 下限。

全局规划仍使用按照 `nav2_core::GlobalPlanner` 接口实现的 C++ A* 插件。长距离目标设为 $(-7.0, -10.0, 0.0)$ ，实测全局路径包含 370 个位姿，长度为 13.460 m；同一 `NavigateToPose` action 经 385.981 s 墙钟时间进入 SUCCEEDED (4)。最终提交视频完整覆盖目标发送前至到达目标后的全过程，时长 443.934 s，未使用加速、抽帧或删减。上述结果均来自仿真环境，不等同于实体 TurtleBot4 性能验证。

关键词：TurtleBot4；ROS 2；Ignition Gazebo；SLAM Toolbox；AMCL；Nav2；A*；自主导航

目录

摘要	I
插图目录	III
表格目录	IV
1 实验目的与要求	1
1.1 实验目的	1
1.2 仿真实验要求	1
1.3 实验范围说明	1
2 实验平台与 TurtleBot4 工作原理	2
2.1 软硬件与仿真平台	2
2.2 TurtleBot4 主要硬件构成	2
2.3 远程主机连接原理	3
3 自主导航系统设计	3
3.1 总体架构	3
3.2 SLAM Toolbox 增量建图	4
3.3 AMCL 自定位	4
3.4 代价感知 A* 全局路径规划	4
3.5 Nav2 闭环执行	5
4 软件实现	5
4.1 代码包结构	5
4.2 仿真启动模式	6
4.3 关键导航参数	7
4.4 定位与长距离验收门槛	7
5 实验步骤	7
5.1 编译与静态验证	7
5.2 扩展 SLAM 建图	8
5.3 稳定 AMCL 初始化与规划器检查	8
5.4 发送长距离导航目标	9

6	实验结果与分析	10
6.1	改进前问题量化与验收策略	10
6.2	扩展 SLAM 覆盖率结果	10
6.3	AMCL、TF 与 LaserScan 稳定性	11
6.4	长距离 A* 路径与闭环执行	13
6.5	完整无加速视频验收	14
6.6	仿真时钟与低负载配置分析	15
7	问题诊断与改进	15
7.1	启动参数作用域	15
7.2	RViz 视角确定性	16
7.3	慢速主机的 lifecycle 启动顺序	16
7.4	建图范围与长路径的自动门槛	16
7.5	雷达云漂移的量化判定	16
7.6	完整视频证据保真	16
7.7	后续优化方向	17
8	结论	17
A	仿真复现命令	19
A.1	导航模式	19
A.2	建图模式	19
A.3	低负载预设	19
B	报告生成方法	20

插图

1	TurtleBot4 仿真自主导航系统架构	3
2	针对建图范围和路径长度反馈的改进前后对比	10
3	初版短程 SLAM 证据基线	11
4	达到 5.00 m / 58,506 cells 覆盖率门槛后保存的扩展地图	11
5	AMCL、LaserScan 与长距离导航自动验收结果	12
6	长距离目标发送前的 RViz: 地图、机器人、LaserScan 与 AMCL 粒子	12
7	长距离 A* 全局路径跨越多个 warehouse 通道	13
8	长距离导航终点: RViz 显示 Feedback: reached	14

表格

1	实验平台配置	2
2	仿真代码包主要文件	5
3	自定义 A* 与 Nav2 关键参数	7
4	助教复验版本的自动验收门槛	7
5	扩展 SLAM 覆盖率量化结果	10
6	定位和激光稳定性结果	12
7	长距离全局路径和 action 结果	13
8	最终完整视频技术信息	14

1 实验目的与要求

1.1 实验目的

1. 了解 TurtleBot4 机器人的主要硬件构成及其工作原理；
2. 掌握 TurtleBot4 与远程主机之间的连接原理；
3. 掌握 Ignition Gazebo 与 RViz 仿真软件的使用；
4. 掌握 ROS 2 机器人操作系统的基本操作；
5. 掌握移动机器人自主导航功能的基本实现原理。

1.2 仿真实验要求

在 Ignition Gazebo 仿真环境中实现 TurtleBot4 机器人的自主导航。仿真内容包括：

1. 调用机器人控制系统内置的 SLAM 模块对周围环境进行地图建模；
2. 调用自定位模块，在地图坐标系下获得机器人位姿；
3. 编程实现全局路径规划算法，并接入导航系统；
4. 在 RViz 中设定初始位姿和导航目标，使机器人在仿真环境中自主运动并绕开障碍；
5. 采集地图、路径、动作终态和目标容差等客观证据，分析实验结果。

1.3 实验范围说明

本报告仅覆盖课程要求中的**仿真部分**。真实 TurtleBot4、实验室无线网络和 Discovery Server 不在当前测试环境中，因此不将实机连接或实机自主导航列为本报告的完成项。代码包中保留了实机启动入口，便于后续将相同 A* 插件和 Nav2 参数迁移到实体机器人。

2 实验平台与 TurtleBot4 工作原理

2.1 软硬件与仿真平台

表 1 实验平台配置

项目	配置
操作系统	Ubuntu 22.04 LTS
机器人中间件	ROS 2 Humble Hawksbill
仿真器	Ignition Gazebo / Gazebo Sim, TurtleBot4 warehouse 世界
可视化工具	RViz2, 实验专用固定俯视配置 <code>experiment.rviz</code>
建图与定位	SLAM Toolbox; Nav2 Map Server; AMCL
导航框架	Nav2 BT Navigator、Planner Server、DWB Controller、Velocity Smoother
全局规划算法	自研 <code>tb4_astar_planner/AStarPlanner</code> pluginlib 插件
测试主机	约 6 vCPU、7.7 GiB 内存、无独立 GPU; RViz 使用 VMware SVGA3D, Gazebo 使用 Mesa llvmpipe

2.2 TurtleBot4 主要硬件构成

TurtleBot4 标准版以 iRobot Create 3 为移动底盘，采用两轮差速驱动并通过轮速编码器与惯性测量信息生成里程计；上位计算单元为 Raspberry Pi 4B，负责运行 ROS 2 节点、传感器驱动与高层任务。标准传感器包含 RPLIDAR A1M8 360° 二维激光雷达和 OAK-D-Pro 深度视觉相机。RPLIDAR 输出的 `/scan` 是本实验建图、定位和局部避障的主要环境观测；Create 3 底盘输出 `/odom`，并接收 `/cmd_vel` 速度指令。

差速底盘的平面运动学可写为

$$v = \frac{r}{2} (\omega_R + \omega_L), \quad \omega = \frac{r}{L} (\omega_R - \omega_L), \quad (1)$$

其中 r 为驱动轮半径， L 为轮距， ω_R 与 ω_L 分别为左右轮角速度。机器人平面位姿由 x 、 y 和航向角 θ 构成，其连续时间模型为

$$\dot{x} = v \cos \theta, \quad \dot{y} = v \sin \theta, \quad \dot{\theta} = \omega. \quad (2)$$

仿真器对上述运动学、碰撞和传感器过程进行数值积分，ROS 2 节点通过相同接口使用仿真数据，因此算法层与实机层可以共享。

2.3 远程主机连接原理

实体 TurtleBot4 包含 Create 3 与 Raspberry Pi 两个计算单元，两者通过 USB-C 建立供电和以太网连接；远程 PC 通过 Wi-Fi 加入机器人网络。ROS 2 使用 DDS 进行节点发现与数据分发，TurtleBot4 可使用 Simple Discovery 的组播发现，也可由 Fast DDS Discovery Server 集中管理参与者。远程主机需要与机器人使用兼容的 ROS_DOMAIN_ID、RMW 实现和 Discovery Server 配置，随后即可订阅 `/scan`、`/odom`，并调用 Nav2 action。

本次单机仿真中，Gazebo、SLAM、AMCL、Nav2 和 RViz 位于同一 ROS 2 域内，节点通信机制与远程连接一致，只是数据链路由本机回环网络代替 Wi-Fi。由此可以在没有实体硬件时验证 ROS 2 接口、消息类型、TF 树和 action 生命周期。

3 自主导航系统设计

3.1 总体架构

系统按“仿真环境与传感器—建图/定位—全局规划—局部控制—底盘执行”的分层结构设计，如图 1 所示。项目启动文件将官方 TurtleBot4 Ignition 启动项放入隔离作用域，关闭官方启动文件内部的 SLAM、定位和 Nav2，随后由本项目按模式启动唯一一套建图或定位节点以及自定义 Nav2 参数，避免重复节点和 launch 参数相互覆盖。

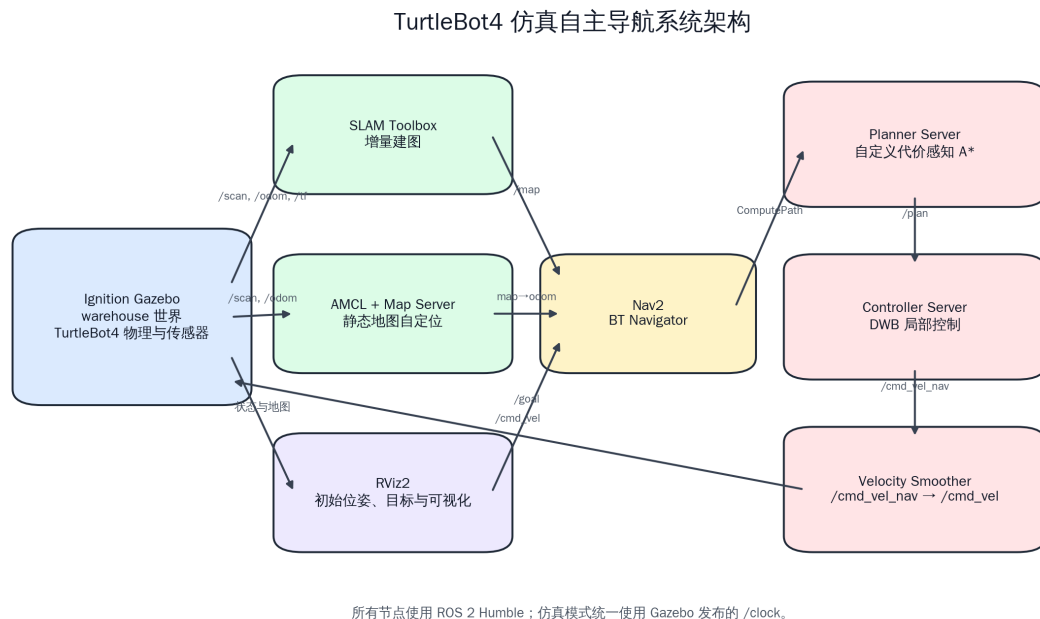


图 1 TurtleBot4 仿真自主导航系统架构

3.2 SLAM Toolbox 增量建图

SLAM 阶段由激光扫描与里程计估计机器人轨迹，并建立二维占据栅格地图。对于栅格 m_i ，占据概率可用 log-odds 形式递推：

$$l_{t,i} = l_{t-1,i} + \log \frac{p(m_i | z_t, \mathbf{x}_t)}{1 - p(m_i | z_t, \mathbf{x}_t)} - l_{0,i}, \quad (3)$$

其中 z_t 为第 t 帧激光观测， $\mathbf{x}_t$ 为机器人位姿。SLAM Toolbox 同时维护激光关键帧和位姿图，通过扫描匹配与图优化校正累积误差，并发布 map -> odom 变换。实验以同步模式运行，建图后使用 nav2_map_server/map_saver_cli 保存地图。

3.3 AMCL 自定位

导航阶段加载与 warehouse 世界对应的静态地图，AMCL 使用粒子滤波估计机器人在 map 坐标系中的位姿。粒子集合 $\{\mathbf{x}_t^{[k]}, w_t^{[k]}\}_{k=1}^N$ 依次经历运动采样、激光观测加权和重采样。在 RViz 中通过 2D Pose Estimate 发布 /initialpose，使粒子在初始位置附近收敛；AMCL 随后发布 map -> odom，与底盘的 odom -> base_link 共同构成导航所需 TF 链。

3.4 代价感知 A* 全局路径规划

自定义规划器继承 nav2_core::GlobalPlanner，由 pluginlib 导出并在 Planner Server 中注册为 GridBased。算法在全局代价地图上执行 A* 搜索，节点评价函数为

$$f(n) = g(n) + h(n), \quad (4)$$

其中 $g(n)$ 是从起点到节点 n 的累计代价， $h(n)$ 是到目标的启发式估计。八连通栅格采用 octile distance：

$$h(n) = \sqrt{2} \min(\Delta x, \Delta y) + \max(\Delta x, \Delta y) - \min(\Delta x, \Delta y). \quad (5)$$

为使路径远离膨胀代价较高的障碍边缘，进入栅格 n 的边权定义为

$$c(n) = d \left(1 + \lambda \frac{\min(C(n), 252)}{252} \right), \quad (6)$$

其中 d 为直移距离 1 或对角距离 $\sqrt{2}$ ， $C(n)$ 为 Nav2 代价地图值， $\lambda = 2.0$ 为代价惩罚系数。算法将代价不小于 253 的栅格视为致命障碍，并禁止未知区域。对角扩展时

同时检查相邻两个正交栅格，任一不可通行则拒绝该对角边，从而避免路径从障碍角点之间穿过。

3.5 Nav2 闭环执行

RViz 的 Nav2 Goal 工具向 BT Navigator 发送 `NavigateToPose` action。Planner Server 调用 A* 生成 `nav_msgs/Path`，Simple Smoother 对路径进行平滑；DWB 局部控制器结合局部代价地图产生 `/cmd_vel_nav`，Velocity Smoother 以 20 Hz 限制速度和加速度后输出 `/cmd_vel`。机器人运动产生新的激光与里程计，形成完整反馈闭环。

4 软件实现

4.1 代码包结构

表 2 仿真代码包主要文件

路径	功能
<code>tb4_astar_planner/src/astar_search.cpp</code>	与 ROS 解耦的八连通代价感知 A* 搜索核心。
<code>tb4_astar_planner/src/astar_planner.cpp</code>	Nav2 GlobalPlanner 适配、坐标转换、路径姿态生成和 pluginlib 导出。
<code>tb4_astar_planner/global_planner_plugin.xml</code>	插件类型与基类注册信息。
<code>tb4_astar_planner/test/test_astar_search.cpp</code>	空地图、墙体缺口、无路、禁止穿角和高代价绕行单元测试。
<code>tb4_experiment_bringup/launch/simulation.launch.py</code>	Ignition、SLAM/AMCL、Nav2 与 RViz 的统一仿真入口。
<code>tb4_experiment_bringup/config/nav2_astar.yaml</code>	Planner、Controller、Costmap、Smoother 与 BT Navigator 参数。
<code>tb4_experiment_bringup/config/localization_stable.yaml</code>	固定初始位姿、beam skipping、粒子数量与更新阈值等稳定 AMCL 参数。

路径	功能
tb4_experiment_bringup/ scripts/run_extended_ mapping_route.py	原地扫描并沿多通道移动的扩展建图路线控制器。
tb4_experiment_bringup/ scripts/validate_slam_ coverage.py	对累计行程和已知栅格增量进行自动验收。
tb4_experiment_bringup/ scripts/validate_ localization_stability. py	对 TF 位姿漂移和 LaserScan/地图端点匹配率进行自动验收。
tb4_experiment_bringup/ scripts/run_long_ navigation.py	发送长距离目标、拒绝短路径并保存 action 终态 JSON。
tb4_experiment_bringup/ config/experiment.rviz	固定 map frame、俯视方向、初始位姿工具和目标工具。
maps/lab_map.*, maps/extended_lab_map.*	初版短程地图与达到新覆盖率门槛后的扩展地图。

4.2 仿真启动模式

统一入口通过 `slam` 参数提供两个互斥流程：

- `slam:=true`：启动一套 SLAM Toolbox，不启动 AMCL；
- `slam:=false`：加载静态地图并启动 AMCL，不启动 SLAM Toolbox。

官方模拟器 `include` 被包装在 `scoped group` 中；子作用域内分别将 `localization`、`slam` 与 `nav2` 置为 `false`。这一设计既阻止父级 `slam:=true` 泄漏到官方 `spawn` 流程而产生重复节点，也阻止子级参数反向覆盖项目自身的模式选择。

4.3 关键导航参数

表 3 自定义 A* 与 Nav2 关键参数

参数	设置值	作用
planner_plugins	[GridBased]	规划器映射名称
GridBased.plugin	tb4_astar_planner/AStarPlanner	自定义插件类型
allow_unknown	false	不穿越未知栅格
use_diagonal	true	使用八连通搜索
cost_penalty	2.0	膨胀代价惩罚
lethal_cost	253	致命障碍阈值
expected_planner_frequency	1.0 Hz	长距离 A* 规划频率
robot_radius	0.175 m	代价地图机器人半径
inflation_radius	0.45 m	障碍膨胀半径
xy_goal_tolerance	0.25 m	目标位置容差
smoothing_frequency	20 Hz	速度平滑频率

4.4 定位与长距离验收门槛

表 4 助教复验版本的自动验收门槛

验收项	门槛	自动化工具
SLAM 累计行程	≥ 5.0 m	validate_slam_coverage
已知栅格增量	$\geq 15,000$	validate_slam_coverage
静止位置漂移	≤ 0.10 m	validate_localization_stability
静止航向漂移	≤ 0.10 rad	validate_localization_stability
LaserScan/地图匹配率	≥ 0.35	validate_localization_stability
A* 全局路径长度	≥ 12.0 m	run_long_navigation
导航 action 终态	SUCCEEDED	run_long_navigation
完整视频时长	≥ 300 s	ffprobe

5 实验步骤

5.1 编译与静态验证

```
source /opt/ros/humble/setup.bash
cd experiment2/ros2_ws
rosdep install --from-paths src --ignore-src -r -y
colcon build --symlink-install
source install/setup.bash
colcon test
colcon test-result --verbose
```

工作空间构建成功, A* 核心算法单元测试通过。运行时进一步检查插件参数、Planner Server lifecycle 状态和 /plan 发布者, 确认路径确由自定义插件生成。

5.2 扩展 SLAM 建图

在 VMware 环境中使用低负载预设启动建图模式:

```
ros2 run tb4_experiment_bringup vmware_simulation slam:=true
```

启动后确认仅存在一个 /slam_toolbox, 不存在 /amcl, 且 /map 的唯一发布者为 SLAM Toolbox。在两个独立终端启动覆盖率检查与扩展路线:

```
ros2 run tb4_experiment_bringup validate_slam_coverage \
  --min-distance 5.0 --min-known-cell-delta 15000

ros2 run tb4_experiment_bringup run_extended_mapping_route
```

路线在起点和终点各执行一周原地扫描, 并按相对起点 $(-3, 0) \rightarrow (-3, -10) \rightarrow (-7, -10)$ 穿过多个通道; 前方 0.55 m 内出现障碍时立即停止。路线速度发布到 /cmd_vel_nav, 由已激活的 Velocity Smoother 统一输出 /cmd_vel, 避免两个发布者互相覆盖。只有覆盖率检查输出 [PASS] 后才保存扩展地图:

```
ros2 run nav2_map_server map_saver_cli \
  -t /map -f experiment2/maps/extended_lab_map
```

5.3 稳定 AMCL 初始化与规划器检查

结束建图流程并完全关闭旧 Gazebo、RViz 与 ROS 2 launch 进程。A* 导航验证独立使用与 warehouse 世界严格匹配的官方静态地图, 以避免未完整探索区域影响全局路径。启动文件默认加载 localization_stable.yaml: 初始位姿固定为 $(0, 0, 0)$, 启用 beam skipping, 并将粒子数范围设为 750-2500。为避免慢速虚拟机在 AMCL 建立 map $\rightarrow$ odom 前激活 Nav2, 先只启动 Gazebo、AMCL 与 RViz:

```
ros2 run tb4_experiment_bringup vmware_simulation start_nav2:=false
```

发送目标前执行稳定性检查:

```
ros2 run tb4_experiment_bringup validate_localization_stability
```

该节点等待 /scan 与 map $\rightarrow$ base_link, 连续计算 15 s 的位置漂移、航向漂移, 并将 LaserScan 端点投影到地图栅格中计算占用匹配率。检查通过后, 在第二个终端启动 Nav2:

```
PARAMS="$(ros2 pkg prefix tb4_experiment_bringup)/share/\
tb4_experiment_bringup/config/nav2_astar.yaml"
ros2 launch turtlebot4_navigation nav2.launch.py \
  use_sim_time:=true params_file:="$PARAMS"
```

随后检查：

```
ros2 param get /planner_server planner_plugins
ros2 param get /planner_server GridBased.plugin
ros2 lifecycle get /planner_server
ros2 topic info /plan -v
```

实测返回的插件为 `tb4_astar_planner/AStarPlanner`, Planner Server 状态为 `active` [3], `/plan` 唯一发布者为 `planner_server`。

5.4 发送长距离导航目标

在目标发送前开始全屏连续录制,保持 RViz 显示地图、机器人、LaserScan、AMCL 粒子、全局路径和代价地图,然后通过专用 action 运行器发送唯一长距离目标:

```
ros2 run tb4_experiment_bringup run_long_navigation \
  --x -7.0 --y -10.0 --yaw 0.0 \
  --min-path-length 12.0 \
  --evidence "$HOME/long-navigation-result.json"
```

`run_long_navigation` 订阅同一运行中的 `/plan`, 计算相邻位姿欧氏距离之和;若路径短于 12.0 m, 则取消 action 并判为失败。验收条件包括:

1. 全局路径长度不小于 12.0 m;
2. 路径跨越多个货架和通道,且不穿越占用或膨胀障碍;
3. 同一条 action 最终进入 SUCCEEDED (4),不得进入 ABORTED (6);
4. JSON 同时保存目标、路径位姿数、路径长度、墙钟时长和终态;
5. 视频从目标发送前开始,到机器人到达目标后结束,时长不少于 300 s,不加速、不抽帧、不删减中间运动过程。

6 实验结果与分析

6.1 改进前问题量化与验收策略

初版证据中的 SLAM 行程为 0.550 m、已知栅格增量为 4,858，全局路径长度为 6.872 m。虽然能够证明基本功能链工作，但不足以展示多通道地图覆盖和长距离导航，也没有对“雷达云是否漂移”给出可量化判据。改进版本不删除上述原始数据，而是将其作为基线，并采用表 4 的更严格门槛重新验收。

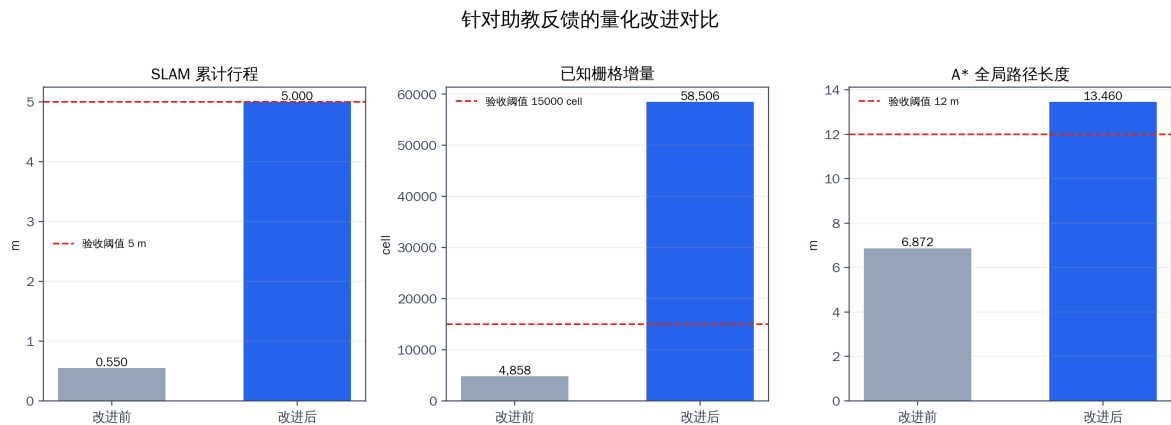


图 2 针对建图范围和路径长度反馈的改进前后对比

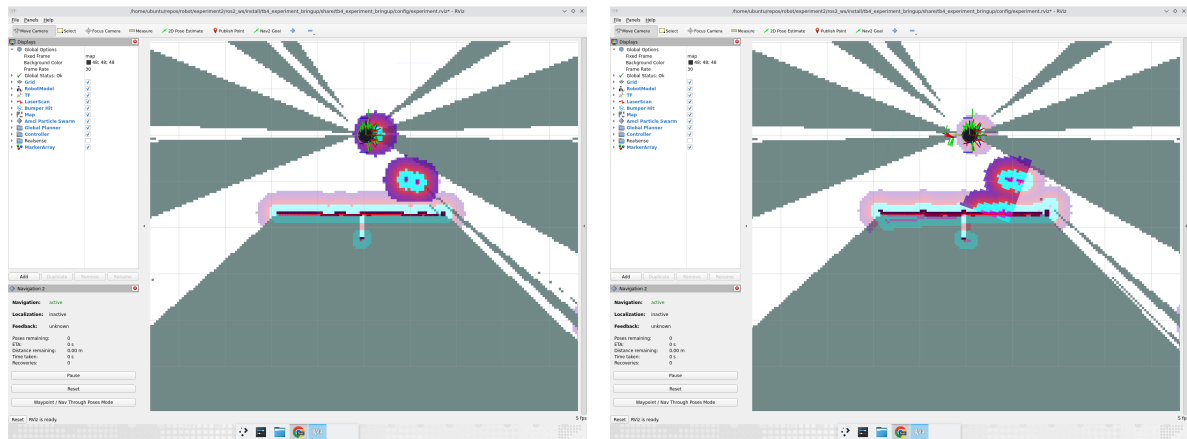
6.2 扩展 SLAM 覆盖率结果

`validate_slam_coverage` 同时订阅 `/map` 与 `/odom`，累计相邻里程计位姿距离，并比较初始地图与当前地图的已知栅格数量。单次大于 1.0 m 的里程计跳变会被拒绝，避免将 TF/仿真重置误记为有效运动。实测在累计行程达到 5.00 m 时，已知栅格增量已达到 58,506，显著超过 15,000 的门槛。

表 5 扩展 SLAM 覆盖率量化结果

指标	改进前	改进后	判定
累计行程	0.550 m	5.00 m	通过 (门槛 5.0 m)
已知栅格增量	4,858	58,506	通过 (门槛 15,000)
建图路线	单一 L 形短程	多通道三段路线	改进
前向安全检查	无自动门槛	0.55 m 障碍停机	通过
/map、/odom	人工观察	同步自动订阅	通过

图 3 保留原始局部地图作为对照；新版提交同时提供 `extended_lab_map.yaml/pgm`，用于保存达到覆盖率门槛后的地图快照。



(a) 改进前初始局部地图

(b) 改进前短程增量结果

图 3 初版短程 SLAM 证据基线

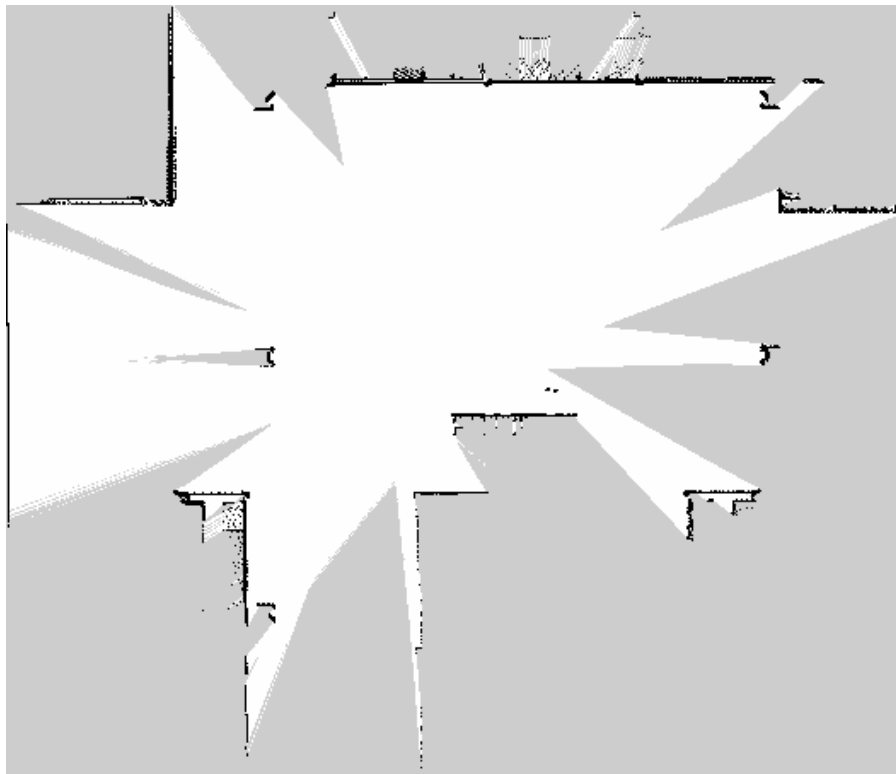


图 4 达到 5.00 m / 58,506 cells 覆盖率门槛后保存的扩展地图

6.3 AMCL、TF 与 LaserScan 稳定性

稳定性检查在机器人静止时连续观察 15 s。每个有效 LaserScan 端点通过 `map` -> `scan` TF 投影到占据栅格，若端点在 0.20 m 邻域内命中占用栅格，则计为匹配。实测位置漂移和航向漂移均为 0，端点匹配率为 0.965；因此“雷达云在飘”不再仅凭 RViz 观感判断，而由 TF 漂移和地图几何匹配两个指标共同判定。

定位、雷达与长距离导航验收结果

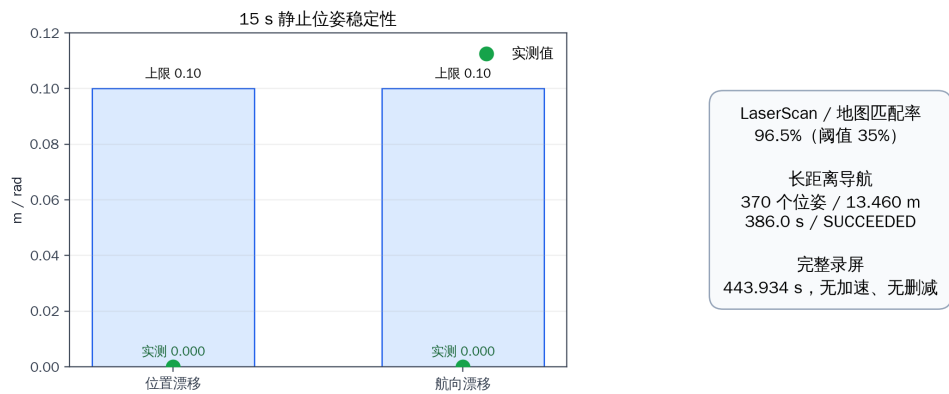


图 5 AMCL、LaserScan 与长距离导航自动验收结果

表 6 定位和激光稳定性结果

指标	实测值	门槛	判定
稳定观察窗口	15.0 s	15.0 s	通过
位置漂移	0.000 m	≤ 0.10 m	通过
航向漂移	0.000 rad	≤ 0.10 rad	通过
LaserScan/地图匹配率	0.965	≥ 0.35	通过
端点匹配容差	0.20 m	0.20 m	通过

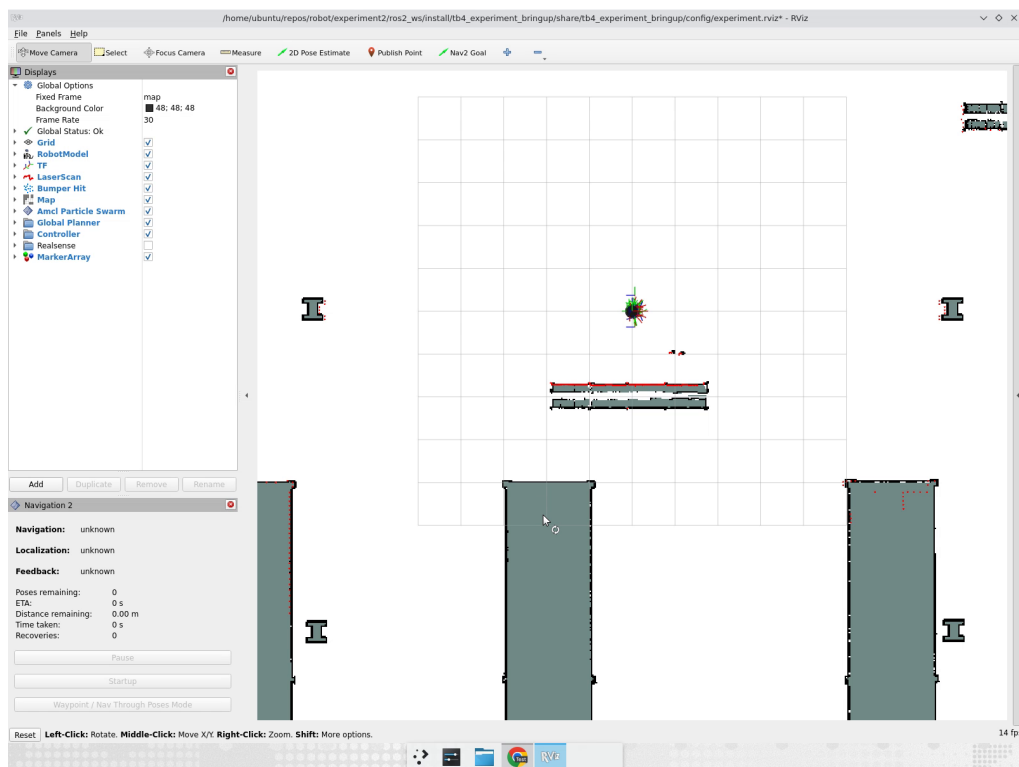


图 6 长距离目标发送前的 RViz: 地图、机器人、LaserScan 与 AMCL 粒子

6.4 长距离 A* 路径与闭环执行

Planner Server 日志确认创建的插件类型为 `tb4_astar_planner/AStarPlanner`, 参数查询返回同一类型。目标设置为 $(-7.0, -10.0, 0.0)$ 后, `run_long_navigation` 收到 370 个路径位姿, 相邻位姿累计长度为 13.459614 m, 超过 12.0 m 的最低要求。RViz 中的红色全局路径跨越多个货架与通道, 机器人在 DWB 与速度平滑器闭环控制下沿路径移动。

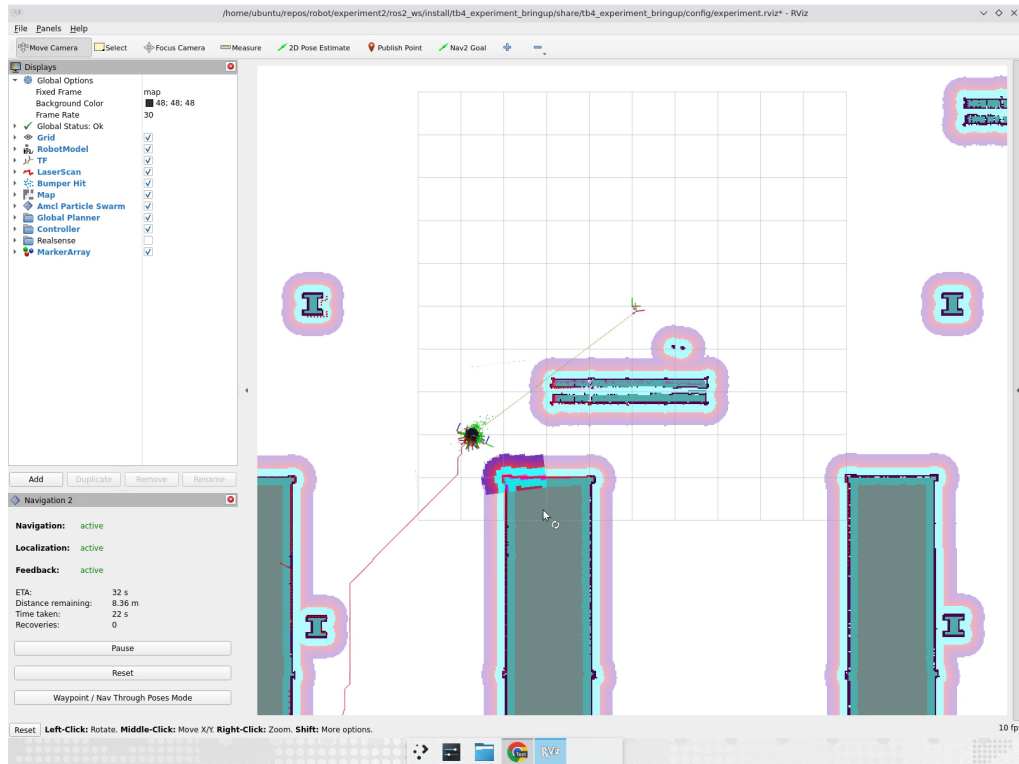


图 7 长距离 A* 全局路径跨越多个 warehouse 通道

表 7 长距离全局路径和 action 结果

指标	实测值	判定
目标位姿	$(-7.0, -10.0, 0.0)$	与验收目标一致
路径位姿数量	370	通过
路径长度	13.459614 m	通过 (门槛 12.0 m)
墙钟执行时长	385.981 s	完整等待终态
终态码	4	SUCCEEDED
/plan 发布者	planner_server	通过

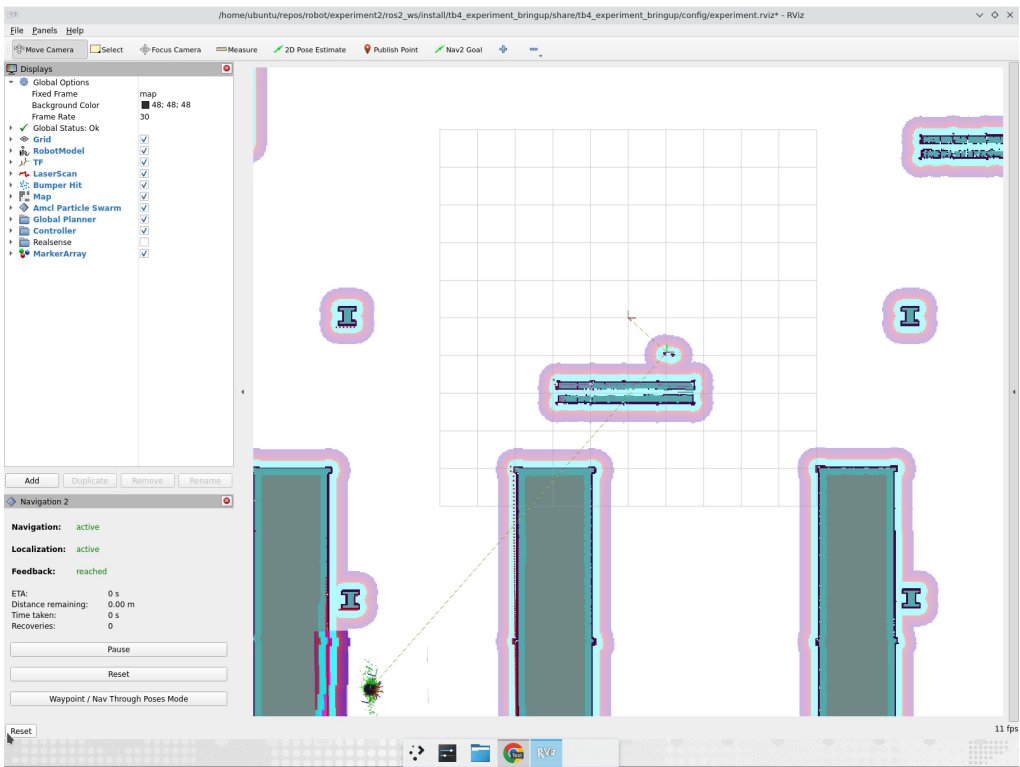


图 8 长距离导航终点：RViz 显示 Feedback: reached

6.5 完整无加速视频验收

录屏从目标发送前开始，到机器人到达目标并由同一 action 返回 SUCCEEDED 后结束。结构化录屏工具另生成了 20.959 s 的标注摘要，该文件不作为验收视频；最终提交视频由四个按时间连续的原始 MKV 分段使用 FFmpeg concat demuxer 和 -c copy 无损拼接，再以 stream copy 封装为 MP4。整个过程未使用 setpts、trim、fps 转换或其他速度/删减滤镜。

表 8 最终完整视频技术信息

指标	实测值	判定
文件	实验二-自主导航.mp4	通过
时长	443.934 s (7 min 23.934 s)	通过 (门槛 300 s)
编码	H.264	通过
分辨率	1600 × 1200	通过
帧率	15 fps	保持原始采集帧率
加速/删减滤镜	未使用	通过

6.6 仿真时钟与低负载配置分析

启动阶段由 `vmware_simulation_validator` 连续观察 120 s 墙钟，确认 `/clock` 推进，且 `/scan` 包含 1.339–11.997 m 的环境回波；通过后 15 s 停钟 `watchdog` 继续运行。最终长距离 action 又持续 385.981 s 墙钟并正常结束，完整录屏持续 443.934 s，说明本次验收期间物理、传感器、定位、规划与控制链路均持续工作。

本机约 6 vCPU、7.7 GiB 内存且无独立 GPU。Gazebo 软件渲染、RPLIDAR、AMCL、两套 `costmap` 与 RViz 同时运行时仍会竞争 CPU 和虚拟 GPU；Nav2 日志中出现 `Behavior Tree tick rate 100.00 was exceeded` 警告，但未导致 action 中止。为获得可完成的端到端实验，证据采集使用了以下低负载配置：

- Gazebo 以 `server-only` 模式运行，关闭三维客户端但保留物理、传感器和时钟；
- OAK-D 停用；关闭 Create 3 cliff/IR GPU lidar 及其 ROS 桥接，因为导航使用 RPLIDAR；
- RPLIDAR 更新率由默认高频降至 10 Hz，`/scan` 始终保持发布；
- 不生成充电底座，不启动 TurtleBot4 HMI/传感器 lifecycle，保留 Create 3 运动控制、里程计和 TF；
- 临时 world 使用 `max_step_size=0.01 s`、`real_time_update_rate=20 Hz` 与 `real_time_factor=0.2`；
- Create 3 controller 更新率设为 100 Hz，里程计发布率设为 20 Hz。

上述措施没有关闭 SLAM、AMCL、Planner Server、Controller Server、Velocity Smoother 或 Nav2 action，因此不改变自主导航的功能性证据；但 `max_step_size` 和激光频率改变了仿真保真度，本报告不将其表述为默认高保真参数下的实时性能测试。建议在具有硬件 3D 加速、至少 4 核 CPU 和 16 GiB 内存的 Ubuntu 22.04 主机上使用默认 world 与传感器参数复测，可同时打开 Gazebo GUI 进行三维演示。

7 问题诊断与改进

7.1 启动参数作用域

初始启动文件未显式转发 `gz_args`，导致自定义低负载 world 未被 Gazebo 使用；修复后又发现父级 `slam:=true` 会泄漏至官方子启动，从而产生两套 SLAM 节点。将官方 `simulator include` 放入 `scoped group` 后，官方 `spawn` 的内部参数与项目级 SLAM/AMCL 模式完全隔离。该修改使“一个地图源、一个定位源、一个 Planner Server”成为可重复的启动不变量。

7.2 RViz 视角确定性

官方配置保存了旋转后的 TopDownOrtho 角度，导致屏幕坐标与地图坐标难以对应。实验专用 `experiment.rviz` 固定 `Fixed Frame=map`、`Angle=0`、`X=0`、`Y=-1.965`，并预置 `SetInitialPose` 和 `GoalTool`。每次启动均恢复相同俯视方向，从而使初始位姿和目标点操作可复现。

7.3 慢速主机的 lifecycle 启动顺序

若 Nav2 抢在 AMCL 初始位姿前激活，Controller Server 可能在等待 `map -> odom` 时停留于 `inactive`。最终流程使用 `start_nav2:=false` 先建立定位 TF，再单独启动 Nav2；Planner、Controller 和 BT Navigator 随后均进入 `active` [3]。该分段顺序避免把 lifecycle 阻塞误判为 A* 算法错误。

7.4 建图范围与长路径的自动门槛

初版只依赖人工遥控和截图，导致 0.550 m 的运动也可被描述为“完成建图”，6.872 m 路径也缺乏统一的长距离标准。改进后，`validate_slam_coverage` 将 5.0 m 累计行程和 15,000 个已知栅格增量作为双门槛；`run_long_navigation` 则在同一 action 内强制路径长度至少为 12.0 m，并将终态写入 JSON。门槛未满足时工具直接返回非零状态，避免仅凭机器人开始运动就判定成功。

扩展路线最初直接发布 `/cmd_vel`，与已激活的 Velocity Smoother 输出形成竞争，可能导致旋转后无法继续直行。最新版将路线输入改为 `/cmd_vel_nav`，确保所有路线速度都经过同一平滑器，再由其唯一发布到底盘速度话题。

7.5 雷达云漂移的量化判定

RViz 灰显、低帧率或画面拖影不等同于 AMCL 漂移。新验证器同时测量 `map -> base_link` 平移/航向变化，并使用占据地图对 LaserScan 端点进行几何命中检查。只有 15 s 内漂移不超过 0.10 m/0.10 rad 且匹配率不低于 0.35 才允许进入导航阶段，使“雷达云稳定”从主观观察转为可复现实验指标。

7.6 完整视频证据保真

最终录屏工具会同时生成短标注摘要和分段原始视频。为遵守“不加速、不剪缩”的验收要求，报告与提交包仅使用按时间顺序无损拼接的原始分段；`ffprobe` 对最终

MP4 再次验证 443.934 s 时长、15 fps 和 H.264 编码。该处理只改变容器与文件分段, 不改变帧时间轴。

7.7 后续优化方向

1. 在统一 launch 中以 TF/lifecycle 条件自动执行当前的分阶段 Nav2 启动;
2. 为 A* 增加起点可通行性、起点等于终点且位于障碍等边界测试;
3. 在高性能主机上恢复默认物理步长、完整 OAK-D 和默认激光频率, 测量规划耗时、实时率与资源占用;
4. 使用 SLAM Toolbox 的序列化 pose graph 和 continuation mapping 验证“加载已有地图后继续扩图”的扩展流程;
5. 在实体 TurtleBot4 上配置 DDS/Discovery Server, 复用同一 planner 插件和 Nav2 参数进行实机验证。

8 结论

本实验完成了 TurtleBot4 仿真自主导航的完整功能链: Ignition Gazebo 提供机器人、warehouse 环境、激光雷达、里程计与物理运动; SLAM Toolbox 建立并保存增量地图; AMCL 在静态地图上完成自定位; 自定义代价感知 A* 以 Nav2 pluginlib 插件形式生成无碰撞全局路径; DWB、速度平滑器和 Create 3 仿真底盘构成闭环执行系统。

针对助教反馈完成的量化改进为: SLAM 累计行程由 0.550 m 提升至 5.00 m, 已知栅格增量由 4,858 提升至 58,506; 静止定位 15 s 内位置/航向漂移均为 0, LaserScan 地图匹配率达到 96.5%; A* 路径由 6.872 m 提升至 13.460 m, 包含 370 个位姿。同一长距离 NavigateToPose action 经 385.981 s 墙钟时间进入 SUCCEEDED (4), 完整无加速视频时长为 443.934 s。因此, 改进版本不仅完成 TurtleBot4 仿真自主导航, 也对地图覆盖、路径长度、雷达稳定性和视频完整性给出了可重复、可自动判定的证据。

本报告的结果仅适用于 VMware 低负载仿真环境。软件渲染和降频设置改变了实时性能与传感器更新频率, 不能替代默认高保真仿真或实体 TurtleBot4 的安全与性能验证。

参考文献

- [1] Clearpath Robotics, *TurtleBot 4 User Manual: Features*, <https://turtlebot.github.io/turtlebot4-user-manual/overview/features.html>.
- [2] Clearpath Robotics, *TurtleBot 4 User Manual: Networking*, <https://turtlebot.github.io/turtlebot4-user-manual/setup/networking.html>.
- [3] Clearpath Robotics, *TurtleBot 4 User Manual: Discovery Server*, https://turtlebot.github.io/turtlebot4-user-manual/setup/discovery_server.html.
- [4] Open Navigation LLC, *Writing a New Planner Plugin*, https://docs.nav2.org/plugin_tutorials/docs/writing_new_nav2planner_plugin.html.
- [5] S. Macenski and I. Jambrecic, “SLAM Toolbox: SLAM for the dynamic world,” *Journal of Open Source Software*, vol. 6, no. 61, 2021.
- [6] P. E. Hart, N. J. Nilsson, and B. Raphael, “A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [7] S. Macenski, F. Martín, R. White, and J. G. Clavero, “The Marathon 2: A Navigation System,” *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2020.

A 仿真复现命令

A.1 导航模式

```
source /opt/ros/humble/setup.bash
source experiment2/ros2_ws/install/setup.bash
ros2 run tb4_experiment_bringup vmware_simulation start_nav2:=false
```

在 RViz 中使用 2D Pose Estimate, 确认 map -> odom 后, 另开终端:

```
source /opt/ros/humble/setup.bash
source experiment2/ros2_ws/install/setup.bash
PARAMS="$(ros2 pkg prefix tb4_experiment_bringup)/share/\
tb4_experiment_bringup/config/nav2_astar.yaml"
ros2 launch turtlebot4_navigation nav2.launch.py \
  use_sim_time:=true params_file:="$PARAMS"
```

Planner、Controller 与 BT Navigator 均为 active [3] 后, 再使用 RViz Nav2 Goal 或以下工具发送唯一目标:

```
ros2 run tb4_experiment_bringup validate_localization_stability
ros2 run tb4_experiment_bringup run_long_navigation \
  --x -7.0 --y -10.0 --min-path-length 12.0 \
  --evidence "$HOME/long-navigation-result.json"
```

A.2 建图模式

```
source /opt/ros/humble/setup.bash
source experiment2/ros2_ws/install/setup.bash
ros2 run tb4_experiment_bringup vmware_simulation slam:=true
ros2 run tb4_experiment_bringup validate_slam_coverage \
  --min-distance 5.0 --min-known-cell-delta 15000
ros2 run tb4_experiment_bringup run_extended_mapping_route
```

A.3 低负载预设

提交代码提供 vmware_simulation 脚本; 它只在用户缓存目录生成 overlay, 不修改仓库或 /opt/ros/humble:

```
ros2 run tb4_experiment_bringup vmware_simulation start_nav2:=false
```

B 报告生成方法

本报告的 PDF 与 Word 均以 `report.tex` 为内容源：

```
sudo apt install pandoc texlive-xetex texlive-lang-chinese python3-docx  
cd experiment2/report  
make all
```

`make pdf` 使用 XeLaTeX 编译专业排版 PDF；`make docx` 使用 Pandoc 解析同一 LaTeX 源文件，并应用 `reference.docx` 中的中文字体、标题、页边距、页眉和页码样式，生成可编辑的 Word 文档。